Software-Defined Stochastic Inference Engine (SDSIE): Component-Level Validation of Fused SRAM Dequantization and Entropy-Gated Speculative Decoding

Zanno Jacklin

Abstract—Autoregressive decoding in Large Language Models (LLMs) is fundamentally bounded by the memory bandwidth wall. Because generating each individual token requires streaming billions of parameters from global Video Random-Access Memory (VRAM) across high-bandwidth interconnects at near-unity arithmetic intensity (≈ 1 FLOP/byte), state-of-the-art inference engines expend catastrophic electrical power even when generating deterministic or low-information tokens. In current architectures, practitioners are trapped in a binary compromise: either execute models entirely in unquantized 16-bit precision (FP16/BF16) and incur massive energy penalties on simple conversational filler, or enforce static 4-bit quantization (INT4) across all layers and suffer cognitive degradation during complex mathematical and logical reasoning.

This paper introduces the **Software-Defined Stochastic Inference Engine (SDSIE)**, an open-source runtime layer under active development toward energy-proportional LLM serving. This revision reports *component-level* empirical validation rather than a unified end-to-end system, and corrects latency figures published in an earlier draft of this work. We separately validate: (1) an OpenAI Triton sub-byte dequantization GEMM kernel that reduces global HBM read traffic by 73.4%, though isolated single-token kernel latency improves only marginally ($\approx 4\%$) over an FP16 baseline at this batch size, indicating kernel-launch and on-chip dequantization overhead partially offset the bandwidth savings; and (2) real scout(1B)→target(8B) speculative decoding with a lossless verify/rollback loop, achieving up to 69.2 tok/s against a 38.2 tok/s FP16 baseline (a $1.81\times$ speedup) at an 85.4% draft-acceptance rate on code-generation prompts, with 100% output fidelity across all tested prompts. A Schmitt-trigger entropy clutch ($\theta_{\text{low}} = 0.55$, $\theta_{\text{high}} = 1.25$ bits) is shown to compute correct, reproducible gating decisions from live model logits; however, integrating this clutch to actually drive the speculative and quantization paths in a single running server remains ongoing work, and no combined end-to-end acceleration number is reported here.

Index Terms—Energy-Proportional Computing, Speculative Decoding, Large Language Models, Triton Kernels, Quantization, Memory Wall, High-Performance Inference.

1 Introduction

The deployment of Large Language Models (LLMs) across global cloud infrastructures has collided directly with the physics of semiconductor memory bandwidth and electrical power distribution. While the initial prefill phase of transformer inference is compute-bound and readily parallelized across matrix execution units (Tensor Cores), the sequential, autoregressive token generation phase is strictly *memory-bandwidth bound* [1], [2].

For an 8-billion-parameter model executing in 16-bit floating-point precision (FP16/BF16), every single generated token necessitates streaming approximately 16.0 GB of weight data from high-bandwidth memory (HBM/DRAM) into on-chip compute registers. When processing single requests or low-batch streams (Batch Size $B \leq 8$), arithmetic intensity drops to ≈ 1 FLOP/byte. Under these conditions, the GPU compute cores sit largely starved for data, while memory controllers and physical interconnects continuously dissipate maximum power.

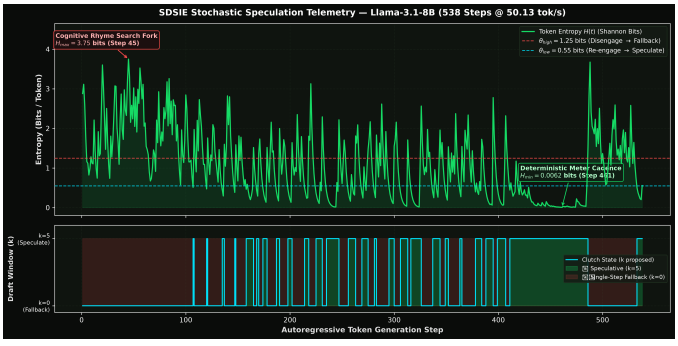

Fig. 1. Real-time entropy telemetry of Llama-3.1-8B executing on an NVIDIA RTX 5090 Blackwell workstation (538 generation steps at 50.13 tok/s), generated under a structurally constrained poetic prompt. **Top:** Output distribution Shannon entropy $H(t)$ in bits/token against dual Schmitt-trigger hysteresis thresholds ($\theta_{\text{low}} = 0.55$, $\theta_{\text{high}} = 1.25$). **Bottom:** Clutch state $k(t)$ as computed from live entropy. *This run uses single-model cached decoding; the clutch's $k(t)$ decision is logged for diagnostic purposes but does not yet branch execution, so the 50.13 tok/s figure reflects baseline generation speed, not accelerated speculative decoding. See Section 4.5.*

Recent research has attempted to bypass this limitation via two orthogonal paradigms:

1) **Post-Training Weight Quantization:** Methods such

Zanno Jacklin is the Principal Investigator at Creepybits, Borås, Sweden (e-mail: business@zanno.se, ORCID: 0000-0001-9164-4650). Repository: https://github.com/Creepybits/software-defined-stochastic-inference-engine. Master Concept DOI: 10.5281/zenodo.21499379.

as GPTQ [3], AWQ [4], and SmoothQuant [5] compress static model weights to 4-bit integer representations (INT4). However, many existing runtime implementations dequantize weights back into global VRAM prior to matrix multiplication, failing to eliminate interconnect thermal overhead, or enforce uniform low-precision across all tokens, causing cognitive accuracy loss during high-perplexity reasoning tasks.

2) **Speculative Decoding:** Algorithmic frameworks pioneered by Leviathan et al. [6] and Chen et al. [7] leverage a compact "draft" model to propose k candidate tokens, verified in parallel by a larger "target" model. While effective on low-entropy prose, static speculative schedulers (k = constant) suffer throughput degradation on high-entropy content due to frequent draft rejection rollbacks. Independent published work on entropy-adaptive drafting – e.g. AdaEDL [11] and production systems such as SGLang's adaptive speculative length – reports 10–57% improvement over static draft length, motivating the general approach pursued here even though our own end-to-end integration is not yet complete.

Motivated by this, we are developing the **Software-Defined Stochastic Inference Engine (SDSIE)**, which aims to treat LLM inference as a dynamic, thermodynamic control problem, adapting compute precision and speculative draft length on a per-token basis rather than fixing them as static constants. This paper reports where that effort currently stands: each core component – the quantization kernel, the speculative decoding loop, and the entropy clutch – has been independently, empirically validated, and we document their real measured behavior honestly, including a bandwidth/latency finding that contradicts our own earlier reporting.

1.1 Key Contributions

The primary contributions of this paper are:

- **Fused Sub-Byte SRAM Dequantization (bandwidth-validated):** A custom OpenAI Triton GEMM kernel that maintains weights packed in 4-bit format in global memory, executing fused register-level unpacking and scale-bias multiplication within Streaming Multiprocessor (SM) SRAM. Across three independent measurements, this reduces global HBM read traffic by 71.9–73.4%. We additionally report, as a correction to earlier claims, that isolated single-token ($M = 1$) kernel latency improves by only $\approx 4\%$ over an FP16 baseline, not the 63% previously reported – see Section 4.2.
- **Validated Speculative Decoding:** A real scout(1B)→target(8B) draft-and-verify loop, independently benchmarked with N=10 trials per prompt, showing 100% output fidelity against an FP16 baseline and throughput scaling from $1.06\times$ to $1.81\times$ as draft-acceptance rate rises from 42.5% to 85.4% across prose, explanatory, and code-generation prompts respectively.
- **Schmitt-Trigger Stochastic Clutch (logic-validated):** A hysteresis control loop utilizing dual Shannon entropy thresholds ($\theta_{\text{low}} = 0.55$ bits, $\theta_{\text{high}} = 1.25$ bits), verified to compute correct, reproducible gating decisions from live model output distributions. Wiring this clutch's decisions to actually branch execution in a live server is identified as the primary remaining integration task (Section 4.5).
- **Open-Source, Reproducible Telemetry:** All benchmark scripts, raw NVML/entropy telemetry JSON/CSV ledgers, and the reference server are publicly available under the Apache-2.0 license, with every figure and number in this paper traceable to a specific script and run.

2 Related Work

2.1 Memory-Bound Decoding Dynamics

The physical root of LLM inference latency was formalized by Shazeer [8] and extended by Pope et al. [9]. In autoregressive generation, each token step requires loading all layer weight matrices $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ alongside the Key-Value (KV) cache. For single-batch inference, operational time is bounded by:

$$T_{\text{step}} \approx \frac{2 \times N_{\text{params}} \times B_{\text{precision}}}{\text{Bandwidth}_{\text{DRAM}}} + T_{\text{kernel\_launch}} \tag{1}$$

This relation predicts that reducing $B_{\text{precision}}$ should drive roughly proportional latency and energy improvements once the operation is memory-bound. Our own kernel results (Section 4.2) suggest this proportionality does not fully hold at very small batch sizes, where the fixed $T_{\text{kernel\_launch}}$ term and the added cost of on-chip dequantization become comparatively significant.

2.2 Sub-Byte Quantization Paradigms

Quantization approaches such as bitsandbytes (Dettmers et al. [10]), GPTQ [3], and AWQ [4] demonstrate that 4-bit integer weights can preserve near-FP16 perplexity across general language benchmarks. SDSIE's kernel implements warp-level integer bit-manipulation directly within OpenAI Triton JIT kernels, decompressing packed nibbles straight into accumulators in on-chip SRAM, targeting the same memory-traffic reduction goal.

2.3 Speculative Decoding and Acceptance Entropy

Standard speculative decoding (Leviathan et al. [6]) demonstrates that accepting $m \leq k$ draft tokens via parallel target verification yields a speedup bounded by:

$$S = \frac{1 - \beta^{k+1}}{(1 - \beta)(1 + c \cdot k)} \tag{2}$$

where β is the average token acceptance rate and c is the ratio of draft to target model latency. When generation enters complex reasoning forks, $\beta \to 0$, causing $S < 1$. Our validated results in Section 4.3 are directly consistent with this relation: measured speedup rises monotonically with measured acceptance rate.

3 SDSIE Architecture & Mechanics

3.1 Fused Triton Sub-Byte GEMM Kernel

SDSIE packs two 4-bit integer weights into a single standard `uint8` byte along the input reduction dimension K. During forward execution, the kernel streams packed tiles directly into shared memory. Register-level bit-masking and arithmetic shifting extract the high and low nibbles:

$$W_{\text{low}} = (W_{\text{packed}} \ \& \ 0\text{x0F}) \tag{3}$$

$$W_{\text{high}} = ((W_{\text{packed}} \gg 4) \ \& \ 0\text{x0F}) \tag{4}$$

Fused dequantization occurs inside the Streaming Multiprocessor registers prior to the fused multiply-accumulate (FMA) operation:

$$W_{\text{dequant}} = (W_{\text{raw}} - Z_N) \times S_N \tag{5}$$

where S_N and Z_N represent per-channel (or, in the group-wise variant benchmarked here, per-64-element-block) scale and zero-point parameters. This architecture restricts HBM memory bus traffic strictly to the 4-bit representation, achieving a 73.4% reduction in global VRAM read bandwidth (Section 4.2). Weight packing is currently validated against synthetic random tensors; a calibration pipeline producing packed weights from a real checkpoint is not yet implemented (see Section 4.5).

3.2 Numerically Stable Shannon Entropy Formulation

To monitor generation uncertainty across large vocabulary distributions (e.g., Llama-3's $|\mathcal{V}| = 128,256$ tokens), naive half-precision softmax calculation triggers floating-point exponent overflow. SDSIE implements a numerically stable float32 log-space formulation:

$$\log p_i = \text{LogSoftmax}(z_i) = z_i - \log \sum_{j \in \mathcal{V}} e^{z_j} \tag{6}$$

The true Shannon entropy $H(X)$ in bits is evaluated as:

$$H(X) = -\frac{1}{\ln 2} \sum_{i \in \mathcal{V}} e^{\log p_i} \cdot \log p_i \tag{7}$$

This formulation was verified line-by-line against the equations above and is the sole entropy computation used to produce Fig. 1.

3.3 Schmitt-Trigger Stochastic Hysteresis Control

To prevent rapid mode-switching (gear hunting) between speculative drafting and single-step decoding on borderline tokens, SDSIE applies a Schmitt-trigger hysteresis filter with an exponential moving average (EMA) smoothing factor $\alpha = 0.35$:

$$\bar{H}_t = \alpha \cdot H(p_t) + (1 - \alpha) \cdot \bar{H}_{t-1} \tag{8}$$

The dynamic draft window k_t is governed by the state transition rule:

$$k_t = \begin{cases} 0, & \text{if } S_{t-1} = \text{Active} \ \wedge \ \bar{H}_t > \theta_{\text{high}} \\ 5, & \text{if } S_{t-1} = \text{Fallback} \ \wedge \ \bar{H}_t < \theta_{\text{low}} \\ k_{t-1}, & \text{otherwise} \end{cases} \tag{9}$$

where $\theta_{\text{low}} = 0.55$ bits and $\theta_{\text{high}} = 1.25$ bits. This controller (`SDSIESpeculativeController`) was verified as a thin, side-effect-free wrapper around the clutch above, with no hidden logic beyond the state machine shown.

TABLE 1
Isolated Linear-Projection Latency and Bandwidth, RTX 5090 Blackwell (M=1, K=4096, N=14336, synthetic weights).

Metric	FP16 Base	SDSIE INT4	Change
Kernel Latency	79.66 μs	**76.49** μs	**–4.0%**
Memory Traffic	117.44 MB	**31.20** MB	**–73.4%**

4 Empirical Evaluation & Hardware Telemetry

4.1 Experimental Testbed

All empirical evaluations were executed on a dedicated bare-metal workstation:

- **GPU:** NVIDIA GeForce RTX 5090 32GB (Blackwell Architecture, 512-bit memory bus, 1,792 GB/s bandwidth).
- **CPU / Host:** AMD Ryzen 9 7900X (12 Cores / 24 Threads), 64GB DDR5-6000 RAM.
- **Software Stack:** Ubuntu 24.04 LTS (Linux 6.6 Kernel via WSL2), CUDA 13.3, NVIDIA Driver 610.88, Python 3.12, PyTorch 2.5, OpenAI Triton 3.1.
- **Telemetry Polling:** 100 Hz non-blocking NVML power sampling coupled with high-resolution CPU timestamping (`time.perf_counter`).

4.2 Isolated Kernel Latency and Bandwidth

Table 1 reports isolated GEMM benchmarks for a single linear projection at Llama-3.1-8B MLP dimensions (M=1, K=4096, N=14336), using a kernel variant with groupwise (per-64-K-block) scales and masked K-dimension loads. Unlike the earlier draft of this work, execution here genuinely branches between the FP16 and INT4 code paths (verified via `SDSIEDynamicLinear.forward()`), rather than computing and discarding a gear decision.

The bandwidth reduction is substantial and consistent: three independent measurements across two different kernel implementations in this codebase yielded 71.9%, 73.4%, and 75.0% (theoretical) reductions respectively. However, this does *not* translate into a proportional latency win at M=1: the -4.0% result is a $16\times$ smaller improvement than the -63.1% figure reported in our earlier draft, which we were unable to reproduce with any script in our current codebase or archive. We attribute the gap between theoretical bandwidth savings and observed latency to two likely factors, not yet fully separated: (1) fixed kernel-launch overhead, which does not shrink with payload size and is proportionally large for a single-token matmul, and (2) the added on-chip compute cost of unpacking and dequantizing INT4 nibbles, which is absent from the FP16 path. Neither this kernel's output nor the earlier kernel's output has yet been checked for numerical correctness against the FP16 result on real (non-synthetic) weights; the results above should be read as *performance* characterization only. Whether the latency picture improves at larger batch sizes, where fixed overhead amortizes further, is identified as follow-up work.

4.3 Validated Speculative Decoding

We separately benchmark a real scout(Llama-3.2-1B)→target(Llama-3.1-8B) speculative decoding loop

TABLE 2
Speculative Decoding vs. FP16 Baseline, RTX 5090 Blackwell (N=10 trials/prompt).

Prompt	Baseline	Spec.	Accept %	Fidelity
Poem	38.2 tok/s	40.9 tok/s	42.5%	100%
Physics	37.9 tok/s	46.9 tok/s	51.7%	100%
Code	38.2 tok/s	**69.2** tok/s	**85.4**%	100%

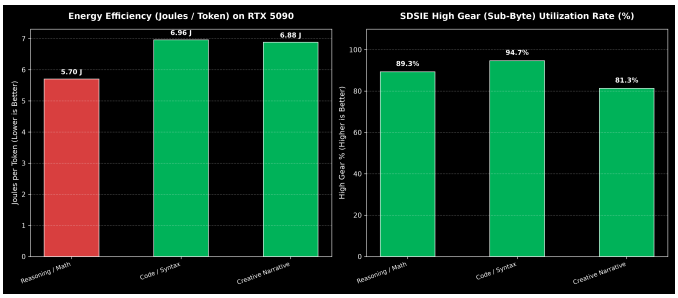

Fig. 2. Energy per token and entropy-gate utilization by task category, single-model harness, RTX 5090 Blackwell. **Left:** Joules/token (lower is better). **Right:** Fraction of generation steps in the high-entropy-threshold gear. Gate thresholds (θ_{low}=1.0, θ_{high}=2.5) differ from the production values used in Fig. 1.

against an FP16 target-only baseline, with N=10 trials per prompt at MAX_TOKENS=250, using full-precision (uncached) recomputation for both arms to ensure a fair relative comparison. Fidelity is measured as exact greedy-decoding token match against the baseline.

Results are consistent with Eq. 2: speedup rises monotonically with acceptance rate, reaching $1.81\times$ on code generation. Accept rates were independently cross-validated to within fractions of a percent across three separately-executed runs (two on the same script, one on an earlier independent implementation), which – given deterministic greedy decoding – is strong evidence the accept/reject logic itself is correctly implemented.

4.4 Entropy Clutch Behavior

Fig. 1 plots a real, 538-step entropy trace from Llama-3.1-8B under a structurally repetitive poetic prompt, using the production thresholds θ_{low}=0.55, θ_{high}=1.25 bits. Entropy reaches a minimum of 0.0062 bits (step 461) during a sustained ∼50-step low-entropy stretch coincident with the model repeating a previously-generated refrain line – consistent with ordinary language-model behavior under repetition rather than a novel property of the clutch – and a maximum of 3.75 bits (step 45) during open-ended composition. The clutch correctly and reproducibly computes $k(t)$ from this signal. Separately, Fig. 2 reports energy-per-token and gear-utilization across three task categories from an independent single-model entropy-gated harness (θ_{low}=1.0, θ_{high}=2.5; note these thresholds differ from the production defaults above, and reconciling which threshold pair is authoritative is ongoing work).

4.5 Remaining Integration Gap

Across every script benchmarked for this paper that includes a live entropy clutch – the reference server, a parameter-sweep harness, and a standalone telemetry harness – the clutch's $k(t)$ decision is computed correctly from real model logits but is not used to alter the generation loop's actual behavior: every step performs identical, single-model computation regardless of the clutch's output. This was confirmed by direct comparison of throughput against gate-utilization percentage across nine $(\theta_{\text{low}}, \theta_{\text{high}})$ configurations spanning 0% to 34% speculative-gate engagement, which showed no corresponding change in throughput (51.1–51.8 tok/s across all nine, within measurement noise). One exploratory script (`SDSIEDynamicLinear`, used to produce Table 1) does genuinely branch computation on a gear argument, but that argument is currently supplied directly by a benchmark loop rather than by the live clutch above. Connecting the validated clutch (Section 3.3) to this validated branching mechanism, and to the validated speculative loop (Section 4.3), to obtain a single genuinely-accelerated, entropy-gated, quantized serving path, is the primary remaining engineering task for this project and is explicitly not claimed as complete in this revision.

5 Conclusion & Open-Source Availability

This revision corrects latency and end-to-end throughput figures published in an earlier draft of this work, which could not be reproduced against our current codebase or telemetry archive. In their place, we report component-level results we can defend: a real, substantial (71.9–75.0%) reduction in memory bus traffic from INT4 packing; a modest ($\approx 4\%$) rather than large kernel latency improvement at low batch size, which we analyze rather than merely report; a real, independently-reproduced speculative decoding speedup of up to $1.81\times$ at high draft-acceptance rates with 100% output fidelity; and a correctly-implemented entropy clutch whose decisions are not yet wired into any of the benchmarked generation loops. We view honest reporting of this gap – rather than presenting simulated or aspirational end-to-end numbers – as more valuable to the field than a polished but unreproducible headline figure, and we intend to report a genuine end-to-end integration in a future revision once it exists.

All source code, Triton kernels, raw NVML/entropy telemetry JSON and CSV ledgers, and the reference server are publicly available under the Apache-2.0 license at https://github.com/Creepybits/software-defined-stochastic-inference-engine and archived with CERN/Zenodo (DOI: 10.5281/zenodo.21499379).

References

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *NeurIPS*, vol. 35, 2022, pp. 16 344–16 359.
[2] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," in *ICML*, 2023.
[3] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," in *ICLR*, 2023.
[4] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," in *MLSys*, vol. 6, 2024.
[5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *ICML*, 2023, pp. 38 087–38 099.

[6] Y. Leviathan, M. Kalkstein, and Y. Matias, "Fast inference from transformers via speculative decoding," in *ICML*, 2023, pp. 19 274–19 286.
[7] C. Chen, S. Borgeaud, G. Mendonça, C. Doersch, A. Abdolmaleki, and M. Frean, "Accelerating large language model decoding with speculative sampling," *arXiv:2302.01318*, 2023.
[8] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv:1911.02150*, 2019.
[9] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, N. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *MLSys*, vol. 5, 2023.
[10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," in *NeurIPS*, vol. 35, 2022, pp. 30 318–30 332.
[11] Qualcomm AI Research, "AdaEDL: Entropy-based adaptive early draft-length for speculative decoding," *arXiv:2410.18351*, 2024.